1372 lines (1372 loc) · 26.9 KB

main Python_Programing / Chapter 2.ipynb Top

Unit II: Collections and Functions (5 Lecture Hours) — Session Plan Table

Lecture	Session Plan – Topics to be Covered	Lecture Date	Actual Delivery – Topics Covered	CO Covered
1	Strings: initialization, operators, indexing, slicing, `split()`			CO2
2	Lists: initialization, methods, operations, list comprehension, nesting			CO2
3	Tuples: init, operations, methods, nesting, List vs Tuple			CO2
4	Sets & Dictionaries: init, methods, operations, nesting, sorting & typecasting			CO2
5	Functions: user-defined, parameters, documentation, scope, recursion			CO2
6	Advanced functions: `map` / `filter` , `lambda` , inner functions, passing collections/functions			CO2
7	Mutable vs Immutable behaviour in functions + Applications of collections + Doubt Session			CO2

Lecture 1 — Strings

1) Initialization

- Strings are **immutable** sequences of characters.
- Create using:
  - Single quotes: `'hello'`
  - Double quotes: `"hello"`
  - Triple quotes (multi-line): `"""hello"""`

In [2]:
```
s1 = 'Python'
s2 = "Data Science"
s3 = """Line1
Line2"""

print(s1)
print(s2)
print(s3)
```

```
Python
Data Science
Line1
Line2
```

2) Operators on strings

- `+` concatenation, `*` repetition
- `in` membership
- comparison uses lexicographic order

In [4]:
```
print("Py" + "thon")      # 'Python'
print("ha" * 3)           # 'hahaha'
print("a" in "cat")       # True
print("abc" < "abd")      # True
```

```
Python
hahaha
True
True
```

In [3]:
```
l =["vibhu", "gautam"]
"vibhu" in l
```

Out[3]: True

3) Indexing (0-based)

In [4]:
```
s = "Python"
print(s[0])   # 'P'
print(s[-1])  # 'n'

print(s[len(s)-1])
# print(s[len(s)])
```

```
P
n
n
```

4) Slicing

Form: `s[start : stop : step]` (stop excluded)

In [17]:
```
s = "Python"
print(s[1:4])    # 'yth'
print(s[:3])     # 'Pyt'
print(s[3:])     # 'hon'
print(s[5:1:-1])   # 'nohtyP'
```

```
yth
Pyt
hon
noht
```

5) `split()`

Splits into list by delimiter (default = whitespace)

In [9]:
```
print("one two three".split())          # ['one','two','three']
```

```
print("2026-02-10".split("-"))
```

```
['one', 'two', 'three']
['2026', '02', '10']
```

Lecture 2 — Lists

1) Initialization

Lists are **mutable**, ordered collections.

In [10]:
```
L = [10, 20, 30]
mixed = [1, "A", 3.5, True]
empty = []
print(L)
print(mixed)
print(empty)
```

```
[10, 20, 30]
[1, 'A', 3.5, True]
[]
```

2) Common methods

- `append(x)`, `extend(iterable)`, `insert(i,x)`
- `remove(x)`, `pop(i)`, `clear()`
- `index(x)`, `count(x)`
- `sort()`, `reverse()`

In [23]:
```
L = [3, 1, 2]
L.append(5)         # [3,1,2,5]
print(L)
L.sort()            # [1,2,3,5]
print(L)
L.pop(2)
print(L)
L.remove(5)
print(L)
L.clear()
print(L)
```

```
[3, 1, 2, 5]
[1, 2, 3, 5]
[1, 2, 5]
[1, 2]
[]
```

3) Operations

- Concatenation, repetition, membership
- indexing/slicing same as strings

In [12]:
```
print([1,2] + [3,4])      # [1,2,3,4]
print([0]*4)              # [0,0,0,0]
```

```
print([0] * 4)          # [0,0,0,0]
print(2 in [1,2,3])     # True
```

```
[1, 2, 3, 4]
[0, 0, 0, 0]
True
```

4) List comprehension

Compact way to build lists.

In [24]:

```
for item in iterator:
    return item*item
```

```
  Cell In[24], line 2
    return item*item
    ^
SyntaxError: 'return' outside function
```

In [25]:

```
squares = [x*x for x in range(1,6)]
evens = [x for x in range(10) if x%2==0]
pairs = [(x,y) for x in [1,2] for y in [3,4,6]]
print(squares)
print(evens)
print(pairs)
```

```
[1, 4, 9, 16, 25]
[0, 2, 4, 6, 8]
[(1, 3), (1, 4), (1, 6), (2, 3), (2, 4), (2, 6)]
```

5) Nesting

List of lists (2D structure).

In [15]:

```
mat = [[1,2,3],[4,5,6]]
print(mat[1][2])   # 6
```

```
6
```

Lecture 3 — Tuples

1) Initialization

Tuples are **immutable** ordered collections.

In [16]:

```
t1 = (1,2,3)
t2 = 1,2,3          # parentheses optional
t3 = (5,)           # single element tuple needs comma

print(t1)
print(t2)
print(t3)
```

```
(1, 2, 3)
(1, 2, 3)
```

```
(5,)
```

2) Operations

- indexing/slicing, `+`, `*`, `in`

In [18]:
```
t = (10,20,30,40)
print(t[1])      # 20
print(t[1:3])    # (20,30)
```

```
20
(20, 30)
```

3) Methods (limited)

- `count(x)`, `index(x)`

In [19]:
```
print((1,2,2,3).count(2))  # 2
print((1,2,2,3).index(2))
```

```
2
1
```

4) Nesting

In [20]:
```
T = ((1,2),(3,4))
print(T[1][0])  # 3
```

```
3
```

5) List vs Tuple

Feature	List	Tuple
Mutability	Mutable	Immutable
Speed	Slower	Faster
Methods	Many	Few
Use case	dynamic data	fixed records / keys

Lecture 4 — Sets & Dictionaries

A) Sets

1) Initialization

Sets are **unordered**, **unique** elements.

In [21]:

In [21]:
```
S = {1,2,3}
S2 = set([1,2,2,3])   # {1,2,3}
empty_set = set()     # {} is empty dict, not set
print(S)
print(S2)
print(empty_set)
```

```
{1, 2, 3}
{1, 2, 3}
set()
```

2) Methods / Operations

- `add` , `remove` , `discard` , `pop`
- `union(|)` , `intersection(&)` , `difference(-)` , `symmetric_difference(^)`

In [24]:
```
A = {1,2,3}
B = {3,4,5}
print(A)
print(B)
print(A | B)   # {1,2,3,4,5}
print(A & B)   # {3}
print(A - B)   # {1,2}
```

```
{1, 2, 3}
{3, 4, 5}
{1, 2, 3, 4, 5}
{3}
{1, 2}
```

B) Dictionaries

1) Initialization

Key–value mapping. Keys must be hashable (immutable types).

In [28]:
```
d = {"name":"Vibhu", "age":38}
d2 = dict(a=1, b=2)
print(d)
print(d2)
```

```
{'name': 'Vibhu', 'age': 38}
{'a': 1, 'b': 2}
```

2) Common operations/methods

- Access: `d[key]` , safe access: `d.get(key, default)`
- Insert/update: `d[key] = value` , `update()`
- Remove: `pop(key)` , `popitem()` , `del d[key]`
- Views: `keys()` , `values()` , `items()`

In [29]:
```
print(d)
d["age"] = 39
print(d)
print(d.get("city", "NA"))    # 'NA'
```

```
{'name': 'Vibhu', 'age': 38}
{'name': 'Vibhu', 'age': 39}
NA
```

3) Nesting

In [30]:
```python
student = {
  "name":"A",
  "marks":{"math":90, "python":95}
}
student["marks"]["python"]   # 95
```

Out[30]: 95

4) Sorting & typecasting

- Sort dict by keys or by values using `sorted()`

In [26]:
```python
a="LKDJFHASDFJHSAD"
sorted(a)
```

Out[26]: ['A', 'A', 'D', 'D', 'D', 'F', 'F', 'H', 'H', 'J', 'J', 'K', 'L', 'S', 'S']

In [32]:
```python
d = {"b":2, "a":5, "c":1}
print(sorted(d))                        # ['a','b','c']
print(sorted(d.items()))                # [('a',5),('b',2),('c',1)]
print(sorted(d.items(), key=lambda x: x[1]))  # by value
```

```
['a', 'b', 'c']
[('a', 5), ('b', 2), ('c', 1)]
[('c', 1), ('b', 2), ('a', 5)]
```

Lecture 5 — Functions

1) User-defined functions

In [33]:
```python
def add(a, b):
    return a + b
```

2) Parameters

- positional, keyword, default, variable-length

In [34]:
```python
def f(a, b=10):
    return a+b

def g(*args):        # tuple of positional args
    return sum(args)

def h(**kwargs):     # dict of keyword args
    return kwargs
```

3) Documentation (docstring)

In [27]:
```
def area_circle(r):
    """Return area of circle for radius r."""
    return 3.14159 * r * r
```

In [30]:
```
area_circle.__doc__
```

Out[30]:
```
'Return area of circle for radius r.'
```

In [31]:
```
sorted.__doc__
```

Out[31]:
```
'Return a new list containing all items from the iterable in ascending order.\n
\nA custom key function can be supplied to customize the sort order, and the\nre
verse flag can be set to request the result in descending order.'
```

4) Scope

- Local, Enclosing, Global, Built-in (LEGB)

In [36]:
```
x = 10
def foo():
    x = 5
    return x
```

In [40]:
```
x=10
print(foo())
print(x)
```

```
5
10
```

5) Recursion

Function calls itself; must have base case.

In [41]:
```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n-1)
```

In [42]:
```
print(fact(4))
```

```
24
```

Lecture 6 — Advanced Functions

1) `lambda`

Anonymous function.

In [ ]:
```
print(square(2))
```

```
4
```

In [33]:
```
def square(x):
    return x*x
```

In [35]:
```
l =[1,2,3,4]
```

In [38]:
```
def square(x):
    return x*x
new_list =[]
for each in l:
    new_list.append(square(each))
```

In [45]:
```
[x*x for x in l]
list(map(lambda x : x*x, l))
```

Out[45]:
```
[1, 4, 9, 16]
```

In [49]:
```
# A simple function to square a number
def square(number):
    return number * number

numbers = [1, 2, 3, 4, 5]

# Map the custom square function to the list
squared_numbers = map(square, numbers)

print(list(squared_numbers))
# Output: [1, 4, 9, 16, 25]
```

```
[1, 4, 9, 16, 25]
```

In [50]:
```
list(map(square, l))
```

Out[50]:
```
[1, 4, 9, 16]
```

2) map()

Apply a function to every element.

In [32]:
```
nums = [1,2,3]
list(map(lambda x: 100/x, nums))  # [1,4,9]
```

Out[32]:
```
[100.0, 50.0, 33.333333333333336]
```

3) filter()

Keep only elements satisfying condition.

In [47]:
```
nums = [1,2,3,4,5]
list(filter(lambda x: x%2==0, nums))  # [2,4]
```

Out[47]: [2, 4]

4) Inner functions + returning functions

In [52]:
```
def outer(msg):
    def inner():
        return msg.upper()
    return inner

f = outer("hello")
print(f)
print(f())   # 'HELLO'
```

```
<function outer.<locals>.inner at 0x0000017E1EE9C540>
HELLO
```

In [54]:
```
outer("hellp")()
```

Out[54]: 'HELLP'

5) Passing collections/functions

In [50]:
```
def apply_twice(f, x):
    return f(f(x))

apply_twice(lambda a: a+1, 5)
```

Out[50]: 7

Lecture 7 — Mutability in functions + Applications + Doubts

1) Mutable vs Immutable behavior

Immutable types: `int, float, str, tuple` **Mutable** types: `list, dict, set`

Key idea:

- If you modify a mutable object inside a function, the caller sees it.
- If you "modify" immutable objects, you actually create a new object.

In [51]:
```
def change_num(x):
    x = x + 1

def change_list(L):
    L.append(99)
```